

2.1 Basic Blocks of a Microcomputer (Easy Explanation)

A **microcomputer** is made of **three main parts**.

Main Parts (Types)

- Central Processing Unit (CPU)
- Memory Unit
- Input / Output (I/O) Unit

1. Central Processing Unit (CPU)

The CPU is the **brain** of the microcomputer. It **executes instructions** and performs **calculations** (addition, subtraction, AND, OR, etc.). In a microcomputer, the CPU is called a **microprocessor**. A microprocessor is usually **one VLSI chip**.

CPU Contains (Types)

- Registers
- Control unit
- Arithmetic Logic Unit (ALU)

Exam line:

CPU executes instructions and performs arithmetic and logic operations.

2. Memory Unit

The memory unit stores **programs (instructions)** and **data**. There are **two main types of memory**.

Types of Memory

- ROM (Read Only Memory)
- RAM (Random Access Memory)

a) ROM (Read Only Memory)

Data can be **only read**, not written. It is **non-volatile**, which means data is **not lost when power is OFF**. It is used to store **fixed programs and data**. An example of ROM data is seven-segment display codes (0–9). ROM is non-volatile and stores permanent programs.

b) RAM (Random Access Memory)

Data can be **read and written**. It is **volatile**, which means data is **lost when power is OFF**. It is used to store **temporary programs and data**. Its contents change during program execution. RAM is volatile and stores temporary data and programs.

3. Input / Output (I/O) Unit

The I/O unit connects the microcomputer to **external devices**. It uses **I/O ports (registers)** and transfers data, status signals, and control signals. Examples of external devices are keyboard, display, printer, and sensors.

I/O unit transfers data between microcomputer and external devices.

Single-Chip Microcomputer vs Microprocessor

Types

- **Single-chip microcomputer**
 - CPU, memory, and I/O are **on one chip**
- **Single-chip microprocessor**
 - Only CPU on one chip
 - Memory and I/O are **separate chips**

Microcontroller

Microcontrollers come from single-chip microcomputers and are used for **dedicated applications**. They usually include timers, ADC, and DAC.

- Applications (Types)

- Cars (automotive systems)
- Home appliances
- Entertainment systems

- Includes (Types)

- Timers
- ADC (Analog-to-Digital Converter)
- DAC (Digital-to-Analog Converter)

Microcontroller examples include PIC microcontroller (8-bit) and Motorola HC11.

2.2 Typical Microcomputer Architecture (Easy Explanation)

Most microcomputers work on **the same basic principle**. Differences are mainly in the number of address bits, number of data bits, and control signals. Understanding **one typical architecture** helps to understand **any microcomputer**.

2.2.1 System Bus

A **system bus** connects the microprocessor, ROM, RAM, and I/O devices. It has **three buses**.

Types of System Bus

- Address Bus
- Data Bus
- Control Bus

1. Address Bus

The address bus carries the **address of memory or I/O location**. Data flows in **one direction only** from microprocessor to memory or I/O, so it is **unidirectional**. It is usually **20 to 32 bits wide**. The number of address lines decides the **maximum memory size**. For example, a 32-bit address bus provides $(2^{32} = 4)$ GB memory and the address range is 00000000_{16} to $FFFFFFFF_{16}$. Address bus is unidirectional and determines memory size.

2. Data Bus

The data bus carries **actual data**. Data flows **both ways**, from CPU to memory during WRITE and from memory to CPU during READ, so it is **bidirectional**. Its size depends on the processor. Examples include Pentium with 64-bit data bus and 68020 with 32-bit data bus.

Exam line:

Data bus is bidirectional and transfers data.

3. Control Bus

The control bus carries **control signals**. It is used to **coordinate and control operations**. It tells memory or I/O about READ or WRITE operations and provides timing and synchronization. Each microprocessor has its own control signals.

Exam line:

Control bus controls and synchronizes system operations.

READ and WRITE Operations

- **WRITE operation**
 - CPU → Memory / I/O
 - Data is stored
- **READ operation**
 - Memory / I/O → CPU
 - Data is fetched

2.2.2 Clock Signals (Easy Explanation)

Clock signals are part of the **control bus**. They provide **timing** for the microprocessor. Every operation in the microprocessor happens **according to the clock**.

The clock is like a **heartbeat**. Each beat tells the CPU **when to do the next step**.

Clock Generator

Some microprocessors have an **internal clock generator**. These need an **external crystal** or an **RC circuit** to set the speed. **Intel 80186** has an internal clock generator. Most microprocessors **do not** have internal clock generators and need an **external clock chip or circuit**.

Clock Frequency

Clock frequency is the **number of clock cycles per second**. The unit is **Hertz (Hz)**. Modern microprocessors work at **more than 2 GHz** (2×10^9 Hz).

Clock frequency determines the speed of the microprocessor.

Clock Cycle

One clock cycle is the time of one clock pulse.

Clock cycle time = $1 / f$, where **f = clock frequency**.

Instruction Execution Time

Instructions take a **fixed number of clock cycles**. An ADD instruction takes **3 clock cycles**.

For a **100 MHz CPU**, the clock cycle is **10 ns**, so ADD time is $3 \times 10 = 30$ ns.

For a **200 MHz CPU**, the clock cycle is **5 ns**, so ADD time is $3 \times 5 = 15$ ns.

Conclusion: Higher clock frequency → **faster execution**.

2.3 Single-Chip Microprocessor

The **microprocessor is the CPU** of a microcomputer. The **power and speed** of a microcomputer depend on clock frequency, number of data pins (word size), number of address pins (memory size), and control pins (I/O capability).

Main Parts Inside a Microprocessor (Types)

- Register Section
- Control Unit
- Arithmetic Logic Unit (ALU)

A microprocessor consists of registers, control unit, and ALU.

2.3.1 Register Section (Easy Explanation)

Registers are **small, fast storage locations** inside the CPU. They store data, addresses, instructions, and results. Register design affects CPU architecture and ease of programming.

Basic Microprocessor Registers (Types)

- Instruction Register (IR)
- Program Counter (PC)
- Memory Address Register (MAR)
- Accumulator / General-Purpose Register (GPR)

1. Instruction Register (IR)

The Instruction Register stores the **current instruction**. The instruction is fetched from memory, stored in IR, decoded, and executed.

Instruction Register stores the instruction being executed.

2. Program Counter (PC)

The Program Counter stores the **address of the next instruction**. It automatically increases after each instruction. It changes when a JUMP instruction is executed or RESET is applied.

- After RESET, PC loads the **first instruction address**
- PC places address on the address bus
- PC size depends on address bus size

👉 **Exam line:** Program Counter holds the address of the next instruction.

3. Memory Address Register (MAR)

The Memory Address Register stores the **address of data** in memory. It acts as a **pointer** to the memory location and is used during data transfer.

👉 **Exam line:** MAR stores the address of data in memory.

4. Accumulator / General-Purpose Register (GPR)

Accumulator (8-bit CPUs)

The accumulator stores **results of ALU operations**. It is used for input from external devices and output to external devices. It can shift or rotate data.

General-Purpose Registers (16- & 32-bit CPUs)

General-purpose registers replace the accumulator. Any GPR can act as an accumulator. They are used to store data, addresses, and results.

👉 **Exam line:** GPRs store data, addresses, and operation results.

Accumulator-Based vs GPR-Based Microprocessors

Accumulator-Based (Types)

- One main register (accumulator)
- All operations use accumulator
- Used in **8-bit CPUs**

Examples include Intel 8085 and Motorola 6809.

GPR-Based (Types)

- Many general-purpose registers
- Faster and flexible
- Used in **16- & 32-bit CPUs**

Examples include Pentium and Motorola 68000 / 68020.

Other Important Registers

General-Purpose Registers (Details)

General-purpose registers store temporary data and reduce memory access, resulting in **faster execution**. Their size is **16 to 32 bits**.

Common Operations (Types)

- Move register ↔ memory
- Move register ↔ register
- Increment / decrement register
- Load data into register

👉 **Exam line:** Using GPRs increases execution speed.

Index Register (Easy Explanation)

An **index register** is used mainly for address calculation, counting, and temporary data storage. It is very useful when working with **tables** and **arrays**.

Uses (Types)

- Address calculation
- Counting
- Temporary data storage

Indexed Addressing Mode

In indexed addressing, the **index register modifies the address** in the instruction. The **effective address** is calculated as:

Effective Address} = Instruction Address + Index Register

This allows easy access to different elements of a table.

👉 **Simple example:** if base address = 1000, index register = 4, then effective address = 1004.

Key Points

- Indexed addressing is available to programmers.
- Index registers are usually **16-bit or 32-bit**.
- In many CPUs, **general-purpose registers can act as index registers**.

👉 **Exam line:** Index register is used to modify addresses for accessing arrays or tables.

Status Register (Easy Explanation)

The status register is also called **Processor Status Word (PSW)** or **Condition Code Register**. It contains several **bits called flags**. Each flag shows the **result or status of an operation**. Flags are used in arithmetic operations, logic operations, and conditional JUMP instructions.

Exam line: Status register contains flags that indicate the result of operations.

Common Status Flags (Types)

- Carry Flag (C)
- Auxiliary Carry Flag (AC)
- Zero Flag (Z)
- Parity Flag (P)

- Sign Flag (S)
- Overflow Flag (OF)

1. Carry Flag (C)

The carry flag is set to **1** if an arithmetic operation produces a carry beyond word size. It is set to **0** if no carry is produced. It is used in **multi-byte arithmetic**.

👉 **Exam line:**

Carry flag indicates overflow beyond the word size.

2. Auxiliary Carry Flag (AC)

The auxiliary carry flag is used in **BCD arithmetic**. It is set if a carry occurs from **lower 4 bits (nibble)** to upper 4 bits. It is important for decimal operations. **Exam line:** Auxiliary carry flag is used in BCD arithmetic.

3. Zero Flag (Z)

The zero flag is set to **1** if the result is **zero**. It is reset to **0** if the result is non-zero.

👉 **Exam line:** Zero flag indicates whether the result is zero.

4. Parity Flag (P)

The parity flag shows the number of **1s** in the result. It may indicate **even parity** or **odd parity**. The type depends on the microprocessor.

👉 **Exam line:** Parity flag indicates even or odd number of 1s.

5. Sign Flag (S)

The sign flag indicates whether the result is **positive** or **negative**. It is based on **MSB (most significant bit)**. If MSB = 1, the result is negative, and if MSB = 0, the result is positive.

👉 **Exam line:** Sign flag indicates the sign of the result.

6. Overflow Flag (OF)

The overflow flag is used in **signed arithmetic**. It is set if the result exceeds the allowed range of word size. It is defined as: 👉 **Exam line:**

Overflow flag indicates signed arithmetic overflow.

Stack Pointer Register (Easy Explanation)

A **stack** is a special part of RAM. It is used mainly in **subroutines** and **function calls**. The stack works on the **LIFO principle**, which means Last In, First Out. 🙌 **Exam line:** Stack is a LIFO memory used for subroutine calls.

Stack Pointer (SP)

The stack pointer stores the **address of the stack**. It is automatically updated during stack operations.

Stack Instructions (Types)

- **PUSH**
- **POP**

PUSH

PUSH writes data into the stack. The stack pointer changes after PUSH.

POP

POP reads data from the stack. The stack pointer changes after POP.

Stack Growth Types (Types)

- Stack accessed from bottom
- Stack accessed from top

Stack Accessed from Bottom

PUSH causes the stack pointer to be incremented, and POP causes the stack pointer to be decremented.

Stack Accessed from Top

PUSH causes the stack pointer to be decremented, and POP causes the stack pointer to be incremented. **Exam line:** Stack pointer increases or decreases depending on stack type.

Word Size Stack operations use 16-bit or 32-bit registers. The stack pointer changes by 2 bytes for 16-bit and 4 bytes for 32-bit operations.

Stack in Subroutines

During a **CALL**, the CPU automatically **PUSHes the return address** onto the stack. During **RETURN**, the CPU **POPs the return address** from the stack. The program then resumes from the original point. **Exam line:** Stack stores return addresses during subroutine calls.

2.3.2 Control Unit (Easy Explanation)

The **control unit (CU)** is like the **manager or boss** of the microprocessor. It **controls and coordinates** all operations inside the CPU.

Exam line: The control unit reads, decodes, and controls execution of instructions.

Main Functions of Control Unit (Types)

- Reads instructions from program memory
- Decodes instructions stored in the Instruction Register (IR)
- Uses the **opcode** (operation code) to decide:
 - What operation to perform
 - Which ALU blocks to activate
- Generates **enable and control signals**
- Sends control signals through the **control bus**
- Responds to control signals from other devices

👉 **Simple idea:** Opcode tells **what to do**, control unit tells **how and when to do it**.

Common Control Signals (Types)

- RESET
- READ / WRITE
- READY
- Interrupt Request (INT / IRQ)

1. RESET Signal

RESET is used to **start or restart** the microprocessor. It loads the **Program Counter (PC)** with a fixed address. The first instruction is executed from that address.

Examples:

Pentium → PC = FFFFFFFF₁₆

Motorola 68000 → PC loaded indirectly from fixed memory locations

👉 Exam line:

RESET initializes the program counter.

2. READ / WRITE Signal

READ/WRITE signals tell whether the CPU is **reading data** or **writing data**. READ → HIGH, WRITE → LOW. Some CPUs use separate READ and WRITE pins.

👉 **Exam line:**

READ/WRITE signal indicates memory or I/O operation.

3. READY Signal

READY signal is used by **slow memory or I/O devices**. If a device is slow, it makes READY **LOW** and the CPU enters **WAIT state**. When the device is ready, READY goes HIGH and the CPU continues execution.

READY signal allows slow devices to communicate with the CPU.

4. Interrupt Request (INT / IRQ)

External devices use this signal to **get CPU attention**. When an interrupt occurs, the CPU pauses the main program, jumps to **Interrupt Service Routine (ISR)**, performs the required task, and returns to the main program. **Exam line:** Interrupt allows external devices to request CPU service.

2.3.3 Arithmetic Logic Unit (ALU) – Easy Explanation

The ALU is the **calculation unit** of the CPU. It performs **arithmetic operations** and **logic operations**. The ALU size equals the **CPU word size**, e.g., a 32-bit CPU has a 32-bit ALU.

Main Functions of ALU (Types)

- Binary addition
- Logical operations (AND, OR, XOR, etc.)
- One's complement
- Shift and rotate operations (left/right through carry)

👉 **Exam line:**

ALU performs arithmetic and logical operations.

2.3.4 Functional Representation of Microprocessors

Simple Microprocessor (Figure 2.9a)

A simple CPU is used for **understanding**, not practical use. It shows basic internal blocks like registers, ALU, control unit, and internal data bus. The **internal data bus** is inside the CPU and **not the system bus**, which connects CPU to memory and I/O.

👉 **Exam line:** Internal data bus is inside the microprocessor.

Pentium Microprocessor (Figure 2.9b)

The Pentium is a **real and complex CPU**.

Key Features of Pentium (Types)

- Two Instruction Pipelines
- Instruction Decode Unit
- Control ROM
- Cache Memory
- Branch Target Buffer
- Registers
- Barrel Shifter
- Bus Unit

1. Two Instruction Pipelines

- **U-pipe** → Executes all integer & floating-point instructions
- **V-pipe** → Executes simple integer instructions

Result: Two instructions can be executed **at the same time**, giving faster performance.

2. Instruction Decode Unit

Converts fetched instructions into internal operations.

3. Control ROM

Contains **microprograms** and controls execution steps for both U-pipe and V-pipe.

4. Cache Memory

Code Cache stores instructions, and **Data Cache** stores data. This makes execution faster.

5. Branch Target Buffer

Stores branch addresses and helps in **fast branching**.

6. Registers

- **Integer register file** → general-purpose registers
- **Floating-point register file** → floating-point registers

7. Barrel Shifter

Performs **fast shift operations**.

8. Bus Unit

Provides **64-bit data bus**, **32-bit address bus**, and **control signals**. It connects CPU to **external memory and I/O**. *Exam line:* Bus unit interfaces the Pentium with memory and I/O devices.

2.4 Program Execution by Conventional Microprocessors

A **conventional microprocessor** refers to older processors like **Intel 8085 (8-bit)** and **Motorola 68000 (16-bit)**. They execute **one instruction at a time**.

Three Basic Steps of Program Execution (Types)

- Fetch
- Decode
- Execute

1. Fetch

The microprocessor **reads the instruction** from **main memory**. The instruction is stored in the **Instruction Register (IR)**.

 *Exam line:* Fetch means reading the instruction from memory into the instruction register.

2. Decode

The **Control Unit (CU)** looks at the instruction and **understands what operation** to perform (ADD, SUB, MOVE, etc.).

 *Exam line:* Decode means translating the instruction to find the instruction type.

3. Execute

The **Control Unit generates control (enable) signals**. These signals activate **registers** and **ALU**, and the actual operation is performed. *Exam line:* Execute means performing the operation using **ALU and registers**.

Example: Adding Two Registers Task: Add register X and Y, store result in Z.

Steps:

Instruction is **fetch**ed into IR. The CU **decodes** the instruction. The CU executes by sending X and Y to the **ALU**, commands ALU to **ADD**, and stores result in **register Z**.

Important point for exam: Control Unit controls everything by generating enable signals.

2.5 Program Execution by Typical 32-bit Microprocessors

32-bit microprocessors like **Intel Pentium** are advanced compared to 8-bit and 16-bit processors.

Extra Features in 32-bit Microprocessors (Types)

- Cache memory → faster data access
- Memory management → run large programs
- Pipelining → faster instruction execution
- Floating-point unit (FPU) → decimal calculations
- Branch prediction → faster decision making

Exam tip: These features increase speed and performance.

2.5.1 Pipelining

Conventional processors face a problem: the processor is **very fast**, but memory is **slow**, so ALU and CU **remain idle** while waiting for data.

Pipelining allows **multiple instructions to be overlapped**. While one instruction is executing, the **next instruction is being fetched**.

Simple meaning: Pipelining is like an assembly line.

Assembly Line Example

A task has 4 steps: A1, A2, A3, A4. Different hardware performs each step. Many tasks are processed **at the same time**. After the pipeline is full, **one task completes every clock cycle**. *Exam line:* Pipelining increases system throughput.

Use of Pipelining in Microprocessors (Types)

- Instruction execution
- Arithmetic operations

Arithmetic Pipelines (Floating-Point Operations)

Floating-point arithmetic is complex because numbers have **Exponent** and **Significand (mantissa)**.

Example 1: Addition

$$x = 0.9234 \times 10^4, y = 0.48 \times 10^2$$

Steps:

Exponents are **different**, so smaller number y is adjusted $\rightarrow 0.0048 \times 10^4$. Add significands $\rightarrow 0.9234 + 0.0048 = 0.9282$. Final result $\rightarrow 0.9282 \times 10^4$.

Example 2: Subtraction

$$x = 0.9234 \times 10^4, y = 0.9230 \times 10^4$$

Steps:

Exponents are **same**, subtract significands $\rightarrow 0.0004$. Result is **not normalized**, so normalize $\rightarrow 0.4 \times 10^1$.

What is Normalization?

A floating-point number is **normalized** when:

$$0.1 \leq |\text{significand}| < 1$$

Exam line: Normalization ensures unique representation of numbers.

Four Steps of Floating-Point Add/Subtract (Types)

- Exponent comparison
- Exponent alignment
- Significand addition/subtraction
- Post-normalization

Floating-Point Pipeline

Each step is handled by **one pipeline segment** using **combinational circuits** and a **barrel shifter** for fast shifting. *Exam line:* A barrel shifter shifts data quickly in one direction.

Real Example

Motorola **68040** uses a **3-stage floating-point pipeline**:

1. Data conversion
2. Execute
3. Normalization

Instruction Pipelines (32-bit Microprocessors)

Instruction pipelining in **32-bit microprocessors** like **Motorola 68020** divides an instruction into **steps**, with each step performed by **different hardware at the same time**. This leads to **higher speed and better CPU utilization**.

Instruction Cycle (5 Basic Activities) (Types)

- Instruction Fetch $\rightarrow$ Get instruction from memory
- Instruction Decode $\rightarrow$ Control Unit understands the instruction

- Operand Fetch (Data Read) → Read required data from registers/memory
- Operation Execution → ALU performs the operation
- Result Routing → Store the result back to register/memory

 *Exam line:* An instruction cycle consists of fetch, decode, operand fetch, execute, and result routing.

Five-Stage Instruction Pipeline (Types)

- Segment 1 → Instruction Fetch
- Segment 2 → Instruction Decode
- Segment 3 → Operand Fetch
- Segment 4 → Operation Execution
- Segment 5 → Result Routing
- **Latches** are placed between segments to store temporary results.

Pipeline Performance

The **first instruction** requires **5 clock cycles**, but after that, **one instruction completes per clock cycle** if all pipeline stages are busy.

 *Exam line:* After the pipeline is filled, one instruction is completed per clock cycle.

Problem in Pipelines: Branch Instructions

Branch instructions like **JMP** or conditional jumps (**IF**, **LOOP**) cause problems because the **next instruction address is unknown**. The CPU must **wait**, the pipeline becomes **idle**, and **extra clock cycles are wasted**.

 *Exam line:* Branch instructions reduce pipeline efficiency.

Example: Conditional Branch

Instruction stream: I1, I2, **I3 (branch)**, I4, I5

The CPU cannot fetch the next instruction after I3 until it is executed, which requires **extra clocks**.

Solution 1: Target Prefetch

When a branch is detected:

- Fetch the **next instruction**
- Fetch the **branch target instruction**

- Store the target in a **buffer**

Advantage: If the branch is taken, the target is ready and the pipeline keeps working.

 *Exam line:* Target prefetch reduces branch delay by prefetching both paths.

Solution 2: Delayed Branch

The branch does **not take effect immediately**, and the **instruction after the branch always executes**.

Original Code Example

```
2000 LDA X
2001 INC Y
2002 JMP 2050
2003 SUB Z
2050 STA W
```

Modified Code with Delayed Branch

```
2000 LDA X
2001 INC Y
2002 JMP 2051
2003 NOP
2004 SUB Z
2051 STA W
```

- **NOP** = No Operation
- Program result is **unchanged**

Pipeline Execution (Delayed Branch)

The CPU fetches the next instruction **before branch executes**, allowing the pipeline to work **without error**, although **NOP wastes time**.

Improved Version (Instruction Reordering)

```
2000 LDA X
2001 JMP 2050
2002 INC Y
2003 SUB Z
2050 STA W
```

Why this is better: NOP is removed, program result is unchanged, and **pipeline efficiency increases**.

Exam line: Instruction reordering improves pipeline efficiency in delayed branch systems.

Important Point

Even though instructions are **fetches earlier**, **execution order remains the same**, and the program gives **correct results**.

Why Delayed Branch Is Important (Types)

- Makes **parallel execution visible**
- Programmer/assembler must handle pipeline effects
- Key feature of **RISC processors**

 *Exam line:* Delayed branch is a key characteristic of RISC architecture.

2.5.2 Branch Prediction Feature

Branch prediction is a **technique to speed up 32-bit microprocessors**. When a branch instruction occurs, the processor **guesses the branch direction before it happens**. This reduces **pipeline delay** by letting the CPU **decide early** which instruction to execute.

Why Branch Prediction Is Needed (Types)

- Branch instructions break the pipeline
- CPU must wait to know:
 - Will it jump?
 - Where will it jump?

Branch prediction helps the CPU **decide early**.

Look-Ahead Execution (Information)

The CPU **looks ahead** at decoded instructions, maintains an **instruction pool**, and executes useful instructions instead of waiting.

 *Exam line:* 32-bit microprocessors use look-ahead execution to improve instruction throughput.

Branch Prediction in Pentium (Types)

Branch Unit (BU) has two parts:

- **BEU** → Branch Execution Unit
- **BPU** → Branch Prediction Unit

How Branch Works in Pentium (Information) The conditional branch instruction is sent to BU, BEU checks the branch condition, calculates the target address, and Pentium fetches instructions from that address.

Speculative Execution (Information)

Speculative execution means the CPU **executes instructions before knowing if the branch is correct**, based on BPU's prediction.

- **If prediction is correct:** Results are written to registers; execution continues normally; high speed is achieved.
- **If prediction is wrong:** Pipeline is flushed, wrong results are discarded, and CPU fetches correct instructions.

 *Exam line:* Speculative execution improves performance but requires pipeline flushing on misprediction.

2.6 Scalar and Superscalar Microprocessors

Scalar Microprocessor (Types)

- Executes **one instruction per clock cycle**
- Has **one pipeline**
- Example: Intel 80486

 *Exam line:* Scalar processors execute one instruction per cycle.

Superscalar Microprocessor (Types)

- Executes **more than one instruction per clock cycle**
- Has **multiple pipelines**

Processor	Pipelines	Instructions per cycle
80486	1	1
Pentium	2	2

 *Exam line:* Superscalar processors improve performance by executing multiple instructions per cycle.

2.7 RISC vs CISC

RISC (Information)

RISC (Reduced Instruction Set Computer) uses **simple instructions for fast execution and high efficiency**.

Features of RISC (Types)

- Hardwired control → little/no microcode, fixed instruction format
- Single-cycle execution → most instructions finish in one clock cycle
- Load-store architecture → memory accessed only via load/store, arithmetic uses registers
- Simple instructions → few addressing modes, fixed length
- Many registers → reduces memory access
- Efficient pipelining → instructions executed in parallel
- Delayed branch support → instruction after jump executes first

 *Exam line:* RISC architecture focuses on speed and efficient pipelining.

RISC Applications (Types)

- Embedded systems
- Image processing
- Robotics
- Graphics
- Instrumentation

Examples: PowerPC (32-bit, 64-bit)

CISC (Information)

CISC (Complex Instruction Set Computer) has a **large instruction set, many addressing modes, and complex instructions.**

Features of CISC (Types)

- Large number of instructions
- Instructions can access memory directly
- Uses microprogrammed control
- Slower clock speed
- Difficult to pipeline

 *Exam line:* CISC uses microprogramming to support complex instructions.

RISC vs CISC (Exam Table)

Feature	RISC	CISC
Instruction set	Small	Large
Instruction format	Fixed	Variable
Control	Hardwired	Microprogrammed
Clock speed	Fast	Slower
Pipelining	Easy	Difficult
Memory access	Load/store only	Allowed in many instructions
Power consumption	Low	Higher

Important Comparison Point (Information)

CISC advantage: fewer instructions per program

RISC advantage: faster execution using higher clock speed, better pipelining, and compiler optimization

Modern Processors (Information)

Original Intel Pentium → CISC

Pentium Pro & later → **RISC for simple instructions, CISC for complex instructions**, giving high performance and backward compatibility.

Exam line: Modern processors combine RISC and CISC features to achieve high performance.